

Basic Data Manipulation with Pandas



by
Dr M Rajasekhara Babu

School of
Computer Science and Engineering
<https://www.rajababu.org>

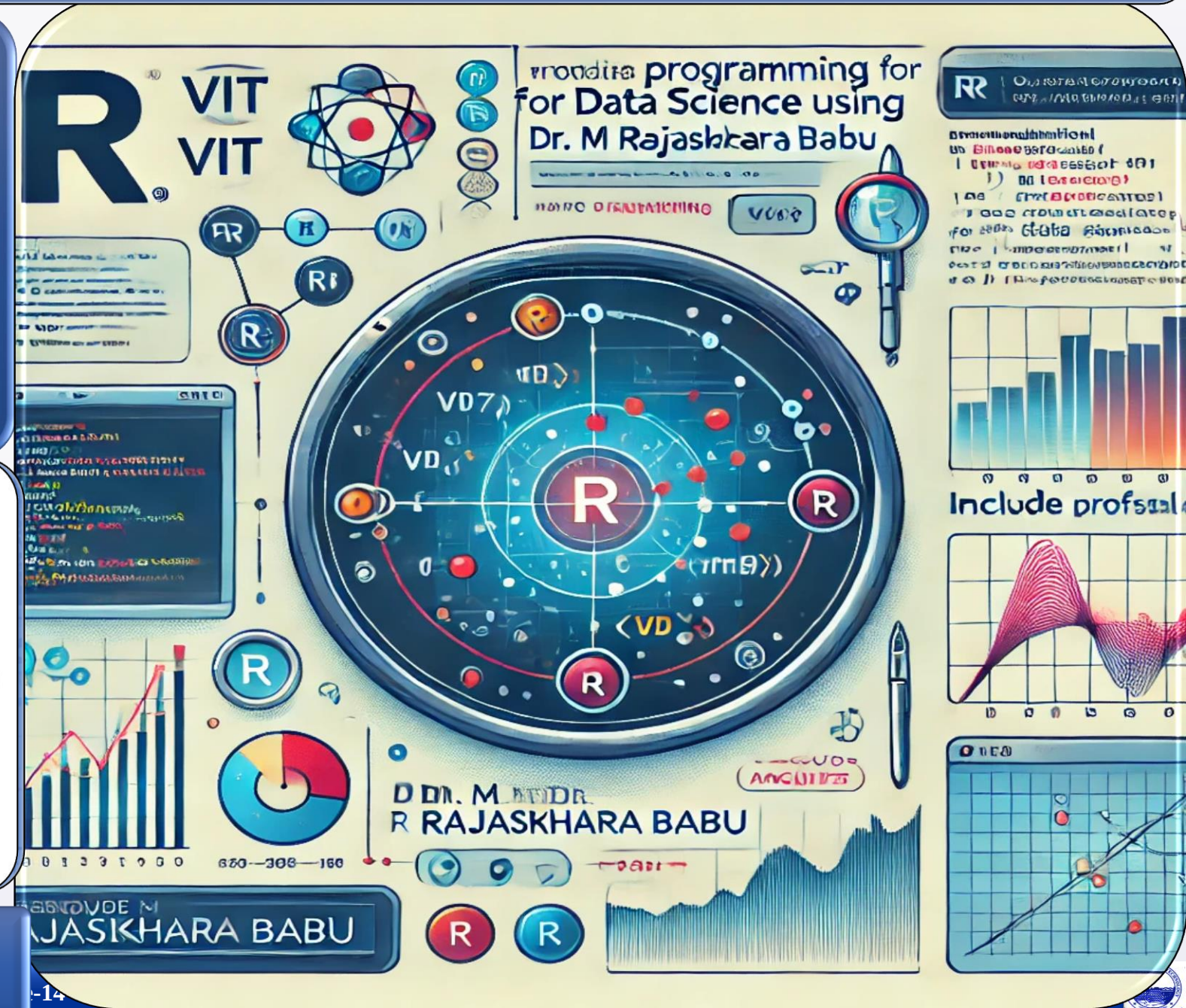


VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Vellore-632014, Tamil Nadu, India



What is Pandas?

Pandas is a powerful Python library used for data cleaning, data analysis, data manipulation, and handling tabular data.

Key Data Structure: DataFrame

A DataFrame is a two-dimensional table with rows and columns — similar to an Excel spreadsheet.

Name	Age	City
Alice	23	New York
Bob	30	London

Why Pandas?



Fast & Efficient

Handles large datasets quickly with optimized C-based operations under the hood.



Integrates Seamlessly

Works natively with NumPy, Matplotlib, Scikit-learn, and the broader Python data science ecosystem.



Flexible & Expressive

Supports complex transformations, grouping, merging, and reshaping with concise, readable syntax.



Basic Data Manipulation with Pandas

Creating a DataFrame

Selecting Columns

Filtering Rows

Adding New Columns

Sorting Data

Calculating Averages





Learning Objectives

After completing this session, students will be able to:

01

Create a Pandas DataFrame

02

Select specific columns from a DataFrame

03

Filter rows based on conditions

04

Add new columns with default values

05

Sort data in ascending or descending order

06

Perform basic statistical calculations



Problem Statement

Create a DataFrame with columns Name, Age, and City for five individuals, then perform the following tasks:

1

Select Columns

Retrieve only Name and Age

2

Filter Rows

Show records where Age > 25

3

Add Column

Append a new Country column

4

Sort Data

Order by Age (Descending)

5

Calculate Average

Find the mean Age

Code

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie',
            'David', 'Emma'],
    'Age': [23, 30, 28, 35, 22],
    'City': ['New York', 'London', 'Paris',
            'Tokyo', 'Sydney']
}

df = pd.DataFrame(data)
print(df)
```

Output

Name	Age	City
Alice	23	New York
Bob	30	London
Charlie	28	Paris
David	35	Tokyo
Emma	22	Sydney

Selecting Specific Columns

Objective: Retrieve only the Name and Age columns from the DataFrame.

Code

```
name_age = df[['Name', 'Age']]  
print(name_age)
```

Output

Name	Age
Alice	23
Bob	30
Charlie	28
David	35
Emma	22

❏ Syntax: `df[['column1', 'column2']]`
— use double brackets to select multiple columns.

Objective: Display only records where Age > 25 using Boolean filtering.

Code

```
filtered_df = df[df['Age'] > 25]
print(filtered_df)
```

Output (Age > 25)

Name	Age	City
Bob	30	London
Charlie	28	Paris
David	35	Tokyo

📄 **Concept — Boolean Filtering:**
`df[condition]` returns only rows
where the condition is `True`.

Adding a New Column

Objective: Add a Country column with the default value "India" for all rows.

Code

```
df['Country'] = 'India'  
print(df)
```

Output

Name	Age	City	Country
Alice	23	New York	India
Bob	30	London	India
Charlie	28	Paris	India
David	35	Tokyo	India
Emma	22	Sydney	India

Sorting Data

Objective: Sort the DataFrame by Age in descending order.

Code

```
sorted_df = df.sort_values(  
    by='Age',  
    ascending=False  
)  
print(sorted_df)
```

Output (Sorted by Age ↓)

Name	Age
David	35
Bob	30
Charlie	28
Alice	23
Emma	22



Syntax:

```
sort_values(by='column_name',  
            ascending=False) — set  
ascending=True for A → Z or  
low → high order.
```

Calculating Average Age

Objective: Find the mean age of all individuals using the `.mean()` method.

Code

```
average_age = df['Age'].mean()  
print(average_age)
```

Formula

$$\text{Average} = \frac{\text{Sum of Ages}}{\text{Number of Individuals}}$$

Calculation

$$\frac{23 + 30 + 28 + 35 + 22}{5} = 27.6$$

Average Age

Across all 5 individuals

```
import pandas as pd

data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Emma'],
    'Age': [23, 30, 28, 35, 22],
    'City': ['New York', 'London', 'Paris', 'Tokyo', 'Sydney']
}
df = pd.DataFrame(data)
print("Original DataFrame"); print(df)

name_age = df[['Name', 'Age']]
print("\nName and Age"); print(name_age)

filtered_df = df[df['Age'] > 25]
print("\nAge > 25"); print(filtered_df)

df['Country'] = 'India'
print("\nCountry Added"); print(df)

sorted_df = df.sort_values(by='Age', ascending=False)
print("\nSorted Data"); print(sorted_df)

average_age = df['Age'].mean()
print("\nAverage Age =", average_age)
```

All Operations at a Glance

Create DataFrame

```
pd.DataFrame()
```

Select Columns

```
df[['col1', 'col2']]
```

Filter Rows

```
df[df['Age'] > 25]
```

Add Column

```
df['Country'] = 'India'
```

Sort Data

```
sort_values()
```

Average

```
mean()
```

Summary Table

Operation	Method / Syntax
Create DataFrame	<code>pd.DataFrame(data)</code>
Select Columns	<code>df[['col1', 'col2']]</code>
Filter Rows	<code>df[df['Age'] > 25]</code>
Add Column	<code>df['Country'] = 'India'</code>
Sort Data	<code>df.sort_values(by='col', ascending=False)</code>
Calculate Average	<code>df['col'].mean()</code>

📌 Pandas makes data manipulation simple, efficient, and suitable for real-world data analysis tasks.



DataFrame

A two-dimensional labeled data structure with rows and columns — the core object in Pandas.



Sorting

Use `sort_values()` with `ascending=True/False` to order data by any column.



Boolean Filtering

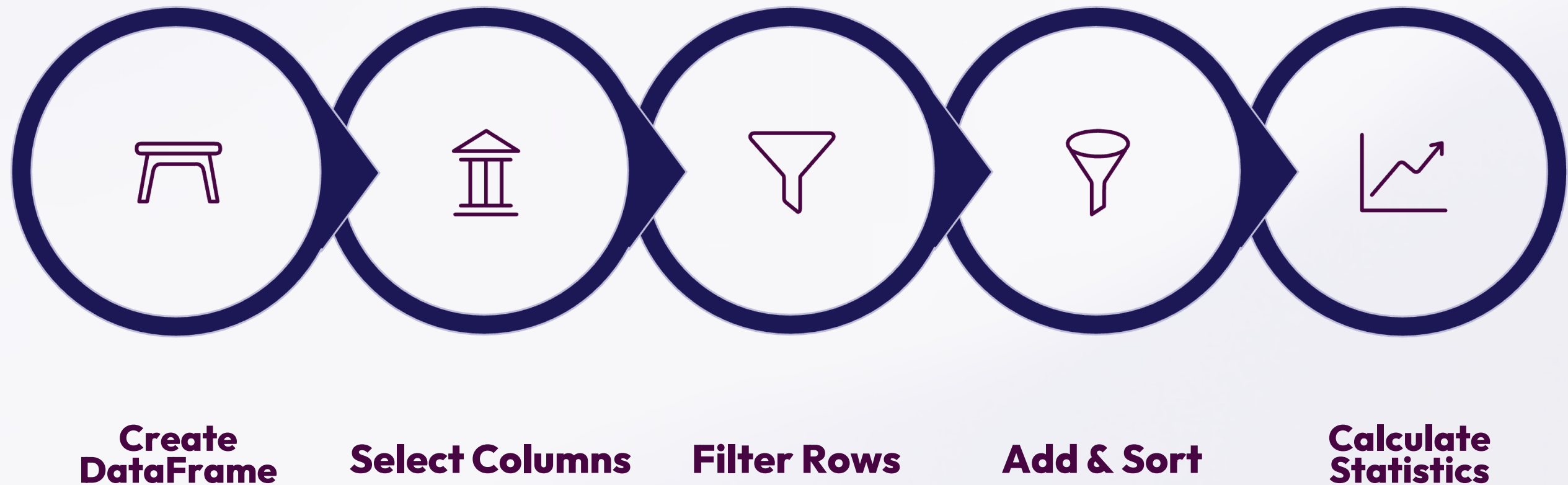
Pass a condition inside `df[...]` to return only rows where the condition evaluates to `True`.



Statistics

Built-in methods like `.mean()` make it easy to compute descriptive statistics on any numeric column.

Data Flow: From Raw Data to Insights



Every Pandas workflow follows this logical progression — from loading raw data into a DataFrame, through selection and filtering, to deriving meaningful statistics and insights.

Practice Exercise

Implement the practice exercise in Python and display all outputs using Pandas.

- ☐ Create a DataFrame with StudentID, Name, and Marks for 5 students
- ☐ Select and display only Name and Marks columns
- ☐ Filter and display rows where Marks > 70
- ☐ Add a Grade column with a default value
- ☐ Sort the DataFrame by Marks
- ☐ Calculate and print the Average Marks

Practice Exercise

Create a DataFrame for 5 students with columns: StudentID, Name, and Marks.

1 **Display Columns**
Show only Name and Marks

2 **Filter**
Show records where Marks > 70

3 **Add Column**
Append a Grade column

4 **Sort**
Order by Marks

5 **Average**
Calculate Average Marks

What's Next?

You've mastered the fundamentals. Here's where to go from here in your Python for Data Science journey:

1

Data Cleaning

Handle missing values, duplicates, and data types

2

GroupBy & Aggregation

Summarize data by categories using `groupby()`

3

Merging DataFrames

Combine datasets with `merge()` and `concat()`

4

Visualization

Plot data with Matplotlib and Seaborn

You're Ready to Manipulate Data with Pandas!

✓ DataFrame Created	✓ Columns Selected
✓ Rows Filtered	✓ Column Added
✓ Data Sorted	✓ Average Calculated

Pandas makes data manipulation simple, efficient, and suitable for real-world data analysis tasks. Keep practicing — the best way to learn is to code!